Cognizant Digital Nurture 5.0

ReactJS Hands-On Exercise 15

React Forms

Objective

The objective of this exercise is to understand and implement **React Forms** by developing a complaint registration application. The application allows employees to enter their name and complaint details through a form. On submitting the form, a unique reference number is generated and displayed in an alert box for future complaint tracking. This exercise demonstrates the implementation of controlled components, form handling, and event handling in React.

Theory

What are React Forms?

React Forms are used to collect user input through various form elements such as textboxes, textareas, checkboxes, radio buttons, and buttons. In React, forms are generally implemented using **controlled components**, where the component state controls the value of form elements.

Controlled Components

A controlled component is a form element whose value is controlled by the React component's state. Whenever the user enters data, the state is updated using event handlers.

Advantages of Controlled Components

- Better control over user input.
- Easy validation of form fields.
- Centralized state management.
- Simplifies form submission.
- Improves code maintainability.

Form Handling in React

React handles forms using event handlers such as:

- onChange()
- onSubmit()
- onClick()

The entered values are stored in the component state and processed when the user submits the form.

React Event Handling

React uses synthetic events for handling user interactions.

Common event handlers include:

- onClick
- onChange
- onSubmit
- onFocus
- onBlur

Prerequisites

- Node.js
- npm
- Visual Studio Code
- ReactJS

Software Requirements

- React
- JavaScript (ES6)
- HTML
- CSS

Implementation Steps

Step 1

Create a React application.

npx create-react-app ticketraisingapp

Step 2

Navigate to the project folder.

cd ticketraisingapp

Step 3

Create the following files inside the **src** folder.

- ComplaintRegister.js
- App.js
- index.js

Step 4

Create a state object to store

- Employee Name
- Complaint
- Reference Number

Step 5

Create the **handleChange()** method to update state whenever the user enters data.

Step 6

Create the **handleSubmit()** method to generate a random reference number and display the confirmation message.

Step 7

Display the form with Employee Name and Complaint fields.

Step 8

Run the application.

npm start

Project Structure

ticketraisingapp

```
├── src
│   ├── ComplaintRegister.js
│   ├── App.js
│   ├── index.js
│   ├── index.css
│
 ├── public
 ├── package.json
└── node_modules
```

Source Code

ComplaintRegister.js

```
import React, { Component } from "react";
class ComplaintRegister extends Component {
  constructor() {
    super();
    this.state = {
      ename: "",
      complaint: "",
```

```
      numberHolder: Math.floor(
        Math.random() * 100
      )
    };
}
handleChange = (event) => {
    this.setState({
      [event.target.name]: event.target.value
    });
};
handleSubmit = (event) => {
    alert(
      "Thanks " +
      this.state.ename +
      "\nYour Complaint was Submitted.\nReference Id is: " +
      this.state.numberHolder
    );
    event.preventDefault();
};
render() {
    return (
      <div
        style={{
          textAlign: "center",
          marginTop: "50px"
        }}
      >
```

```
<h1
  style={{
    color: "red"
  }}
>
  Register your complaints here!!!
</h1>
<form
  onSubmit={this.handleSubmit}
>
  <table
    align="center"
  >
    <tbody>
      <tr>
        <td>
          Name
        </td>
        <td>
          <input
            type="text"
            name="ename"
            value={this.state.ename}
            onChange={this.handleChange}
            required
          />
        </td>
      </tr>
```

```
            <tr>
              <td>
                Complaint
              </td>
              <td>
                <textarea
                  rows="4"
                  cols="25"
                  name="complaint"
                  value={this.state.complaint}
                  onChange={this.handleChange}
                  required
                >
                </textarea>
              </td>
            </tr>
            <tr>
              <td>
              </td>
              <td>
                <button>
                  Submit
                </button>
              </td>
            </tr>
          </tbody>
        </table>
      </form>
```

```
        </div>
      );
    }
}
export default ComplaintRegister;
```

App.js

```
import React from "react";
import ComplaintRegister from "./ComplaintRegister";
function App() {
  return (
    <div>
      <ComplaintRegister />
    </div>
  );
}
export default App;
```

---

index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
const root = ReactDOM.createRoot(
  document.getElementById("root")
);
root.render(
  <React.StrictMode>
```

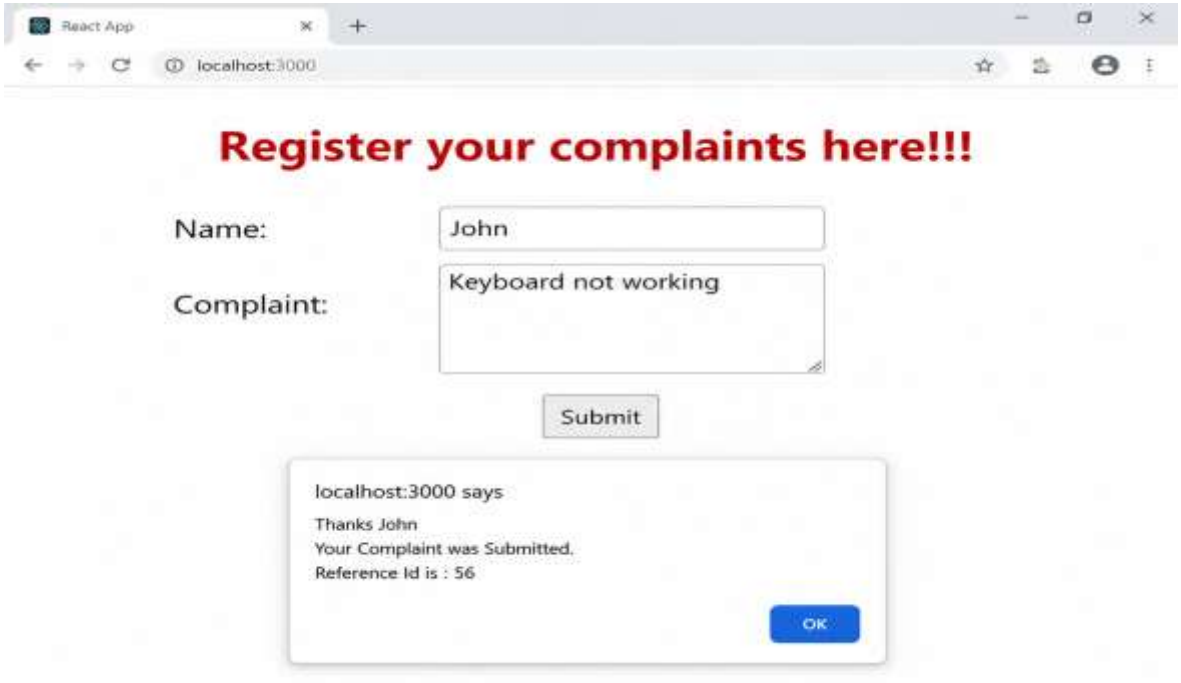
```
    <App />
  </React.StrictMode>
);
```

Output Screenshot:

When the application starts, the browser displays a complaint registration form with the title **"Register your complaints here!!!"**.

The form contains:

- Employee Name textbox
- Complaint textarea
- Submit button



The reference number is generated automatically for every submission, similar to the expected output shown in the exercise document.

Learning Outcome

After completing this exercise, I learned how to:

- Create forms in React.

- Implement controlled components.
- Handle user input using onChange().
- Handle form submission using onSubmit().
- Manage component state.
- Generate dynamic values using JavaScript.
- Display confirmation messages using alert boxes.

Conclusion

This exercise provided practical experience in developing **React Forms** using controlled components. The complaint registration application demonstrated how user input can be managed through component state, how event handlers such as onChange() and onSubmit() are used to process form data, and how a unique reference number can be generated for each complaint. The exercise strengthened the understanding of form handling, state management, and event-driven programming in React, which are essential concepts for building interactive web applications.